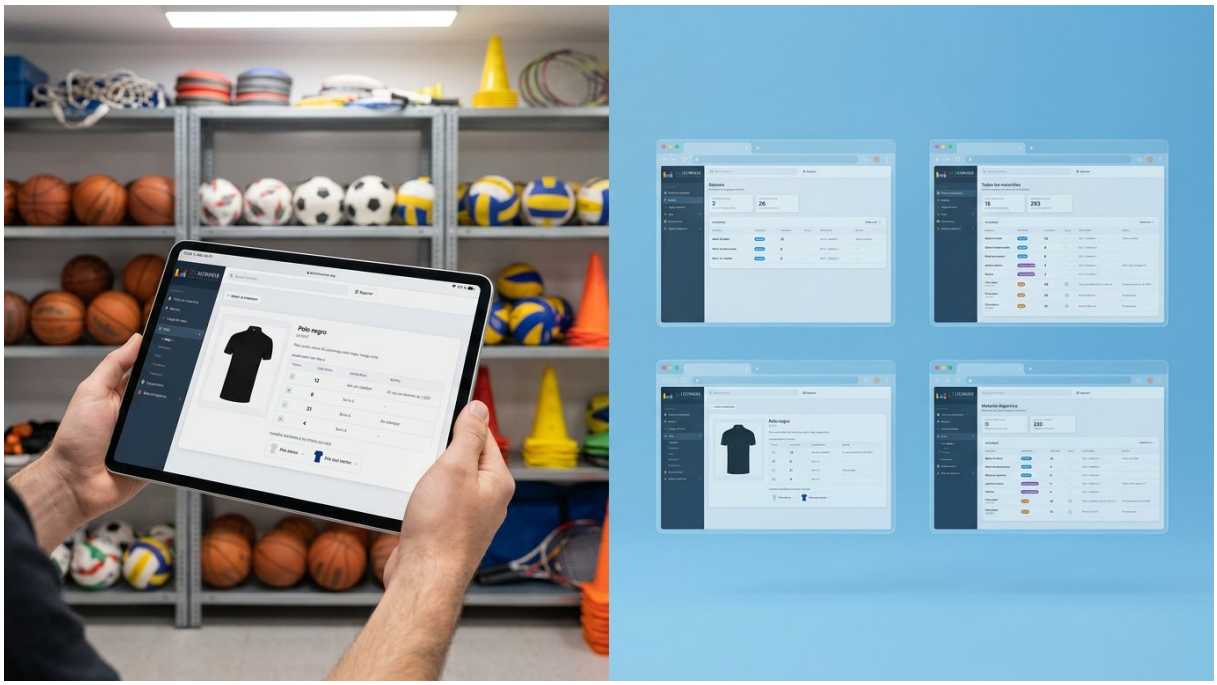


Inventario del Gimnasio

IES Lacimurga · Constantia Iulia

Eva Gallardo Romero

23 de octubre de 2026

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **2**
3.1 Inventario . . . 3
3.2 Ubicación . . . 3
3.3 Etiquetas QR . . . 3
3.4 Avisos . . . 3

4 Requisitos del sistema **4**
4.1 Requisitos funcionales . . . 4
4.2 Requisitos no funcionales . . . 5

5 Documentación técnica **6**
5.1 carga.js . . . 6
5.2 csv.js . . . 7
5.3 datos.js . . . 8
5.4 app.js . . . 9
5.5 qr.js . . . 10

Enlace al repositorio de GitHub

La versión oficial, desde la que se publica la web que usa el instituto, está en la organización del centro: IESLACIMURGA/inventario-gimnasio. Mi copia de portfolio está en evagr11/inventario-gimnasio-estatico.

La web está publicada en ieslacimurga.github.io/inventario-gimnasio y la documentación del código generada con JSDoc, en evagr11.github.io/inventario-gimnasio-estatico/docs.

1 Descripción general

Inventario del Gimnasio es la web que usa el departamento de Educación Física del **IES Lacimurga**, mi antiguo instituto, para saber qué material hay en el almacén: cuántas unidades, de qué talla, en qué estado y en qué estantería está. Cada estantería, armario o caja lleva una **etiqueta QR**; al escanearla con el móvil se abre su página con todo lo que contiene. Es un **proyecto real** y lo hice de forma desinteresada.

Las condiciones eran claras: **coste cero**, **seguro** y que los **datos de los alumnos no salieran del centro**. Diseñé primero los wireframes en **Figma** y, tras valorar una primera versión con servidor y préstamos, la simplifiqué a una **web estática**: los profesores mantienen el inventario en una hoja de **Google Sheets** que ya saben usar, la hoja se publica en solo lectura como CSV y la web la descarga y calcula todos los totales. Sin préstamos ni cuentas no hace falta guardar ningún dato personal, y Google Sheets aporta gratis los permisos (solo editan los profesores) y el historial de cambios.

La hoja tiene cinco pestañas: **Materiales**, **Existencias** (cuántas unidades hay de cada material, talla, ubicación y estado), **Categorías**, **Estados** y **Ubicaciones**. Las cantidades se escriben una sola vez y la web suma el resto, así que los totales siempre cuadran.

2 Diagrama de clases

El código está en módulos ES sin dependencias. `carga.js` descarga las pestañas, `csv.js` las convierte en objetos, `datos.js` construye el modelo con los totales y `app.js` pinta la vista que indica la dirección (`#/...`). Los códigos QR los genera `qr.js`, programado desde cero.

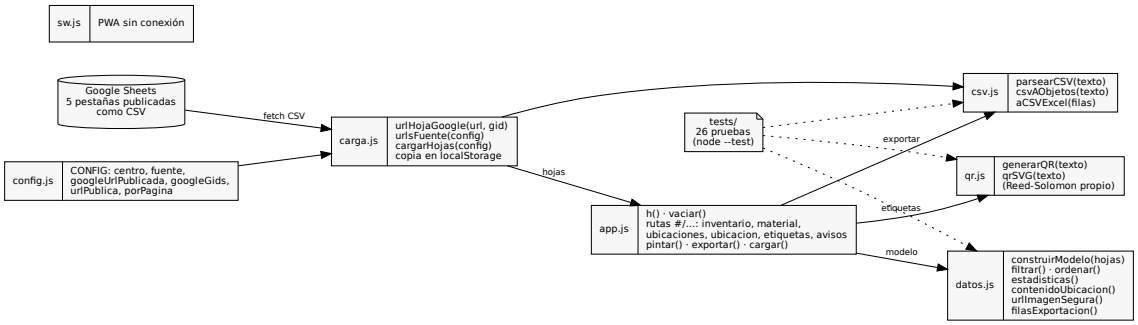


Figura 1: Módulos del Inventario y flujo de los datos

3 Diseño

Panel lateral oscuro con las categorías, el almacén y las etiquetas, y el contenido sobre fondo claro: tarjetas con los totales, chips de colores para categorías y estados y una tabla con buscador,

filtros y orden. En el móvil el menú pasa a un panel que se abre con el botón y la tabla se convierte en fichas. Tiene estilos propios para imprimir.

3.1 Inventario

Totales de materiales, unidades, categorías y ubicaciones, las unidades no disponibles por estado y la lista de materiales paginada. El buscador ignora tildes y mayúsculas y busca por nombre, talla, ubicación o código; se abre con la tecla `/`.

3.2 Ubicación

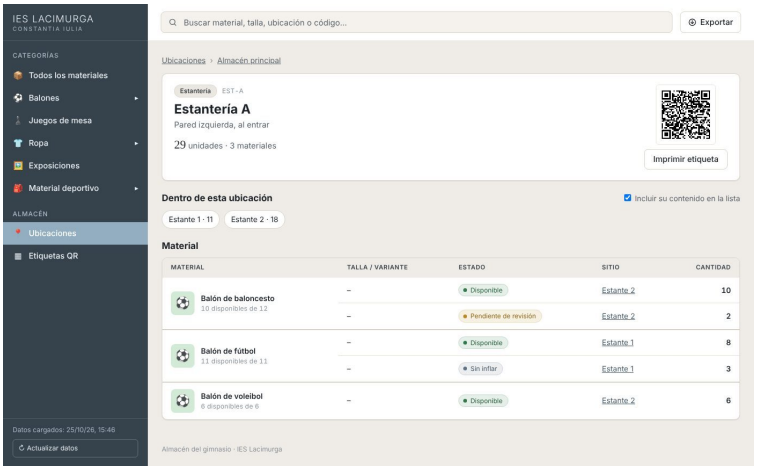
Lo que se ve al escanear un QR: la ruta (almacén, estantería, estante o caja), su código QR, las sububicaciones y el material que contiene con su talla, estado y cantidad.

3.3 Etiquetas QR

Se eligen las ubicaciones y el tamaño, y se imprimen todas las etiquetas a la vez.

3.4 Avisos

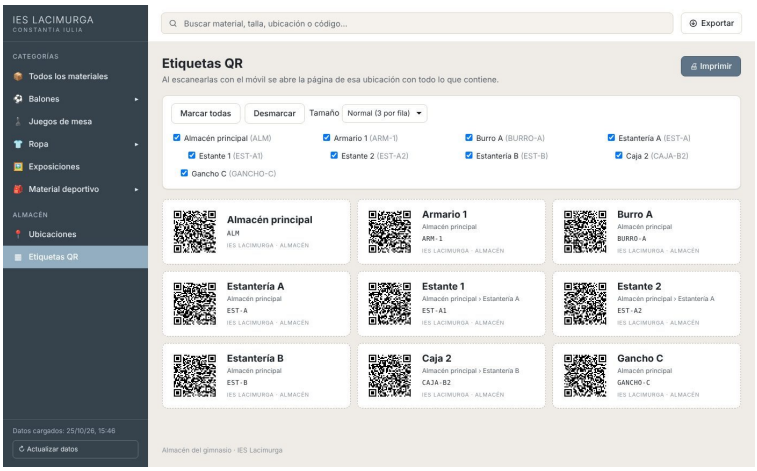
Si en la hoja hay un error (un material que no existe, una cantidad negativa, una fila repetida...), la web no se rompe: lo indica con el número de fila.



3.2. Ubicación



3.1. Inventario en el móvil



3.3. Etiquetas QR

Figura 2: Capturas del Inventario con los datos de ejemplo

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La web debe leer las cinco pestañas de la hoja de Google Sheets publicada como CSV.
RF02	Debe calcular el total y las unidades **disponibles** de cada material, por talla, por estado y por ubicación.
RF03	Cada estado debe indicar si sus unidades cuentan como disponibles.

ID	Requisito funcional
RF04	Debe haber un **buscador** sin tildes ni mayúsculas y filtros por categoría, estado y disponibilidad.
RF05	Filtrar por una categoría debe incluir sus **subcategorías**.
RF06	Las ubicaciones se deben poder anidar, y la página de una ubicación debe incluir lo que hay en las de dentro.
RF07	Cada ubicación debe tener su **código QR** y se deben poder imprimir las etiquetas de varias a la vez.
RF08	Los errores de la hoja deben mostrarse como **avisos** con su fila, sin romper la web.
RF09	El inventario se debe poder **exportar a Excel** e imprimir.
RF10	Sin conexión, la web debe mostrar la **última copia** descargada indicando su fecha.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	**Coste cero**: web estática en GitHub Pages, sin servidor ni base de datos.
RNF02	No se guarda **ningún dato personal** de alumnos.
RNF03	Seguridad: solo lectura, política CSP, todo el texto se inserta como texto (sin `innerHTML`), imágenes solo de rutas propias o `https` y exportación protegida contra inyección de fórmulas.
RNF04	JavaScript con **módulos ES** y sin dependencias externas, incluido el generador de QR.
RNF05	Instalable como **PWA**.
RNF06	**26 pruebas automáticas** con el test runner de Node.
RNF07	El código debe tener comentarios **JSDoc**.

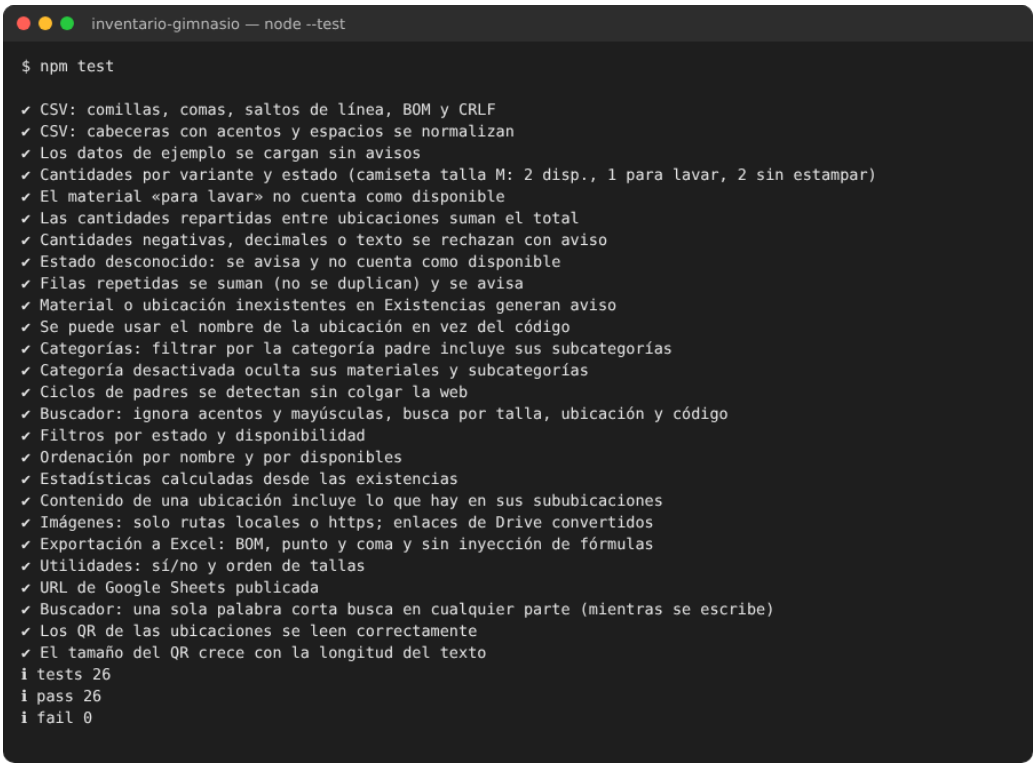


Figura 3: Las 26 pruebas pasando

5 Documentación técnica

Este anexo agrupa la documentación por módulos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Es un proyecto grande, así que se documentan las funciones principales; el resto está en la documentación JSDoc.

5.1 carga.js

5.1.1 Ubicación

`js/carga.js`

Descripción general:
Descarga las pestañas y guarda una copia para usarla sin conexión.

5.1.2 Funciones

urlHojaGoogle **Descripción:** Comprueba que el enlace es de «Publicar en la web» y construye la dirección del CSV de una pestaña.

Código comentado:

```
export function urlHojaGoogle(urlPublicada, gid) {
  const u = String(urlPublicada || '').trim();
  const m = u.match(/^https:\/\/docs\.google\.com\/spreadsheets\/d\/e\/([\w-]+)\/pub/);
  if (!m) throw new Error('El enlace de Google Sheets de config.js no parece un enlace de «Publicar en la web».');
  if (!/^\d+$/.test(String(gid ?? '').trim())) throw new Error('Falta algún número gid en config.js.');
  return `https://docs.google.com/spreadsheets/d/e/${m[1]}/pub?gid=${String(gid).trim()}&single=true&output=csv`;
```

```
}
```

cargarHojas **Descripción:** Descarga las cinco pestañas a la vez. Si todo va bien guarda una copia; si falla, usa la copia guardada de la misma hoja.

Código comentado:

```
export async function cargarHojas(config) {
  const origen = config.fuente === 'google' ? `google:${config.googleUrlPublicada}` : 'demo';
  try {
    const urls = urlsFuente(config);
    const textos = Object.fromEntries(await Promise.all(
      HOJAS.map(async (h) => [h, await descargar(urls[h])])));
    guardarCache(textos, origen);
    return { hojas: aObjetos(textos), fecha: new Date(), desdeCopia: false };
  } catch (error) {
    const copia = leerCache();
    if (copia && copia.origen === origen) {          // solo si es de la misma hoja
      return { hojas: aObjetos(copia.textos), fecha: new Date(copia.fecha), desdeCopia: true, error };
    }
    throw error;
  }
}
```

5.2 csv.js

5.2.1 Ubicación

`js/csv.js`

Descripción general:
Lector de CSV propio y exportación para Excel.

5.2.2 Funciones

parsearCSV **Descripción:** Lee el CSV carácter a carácter: admite comillas, comas y saltos de línea dentro de las celdas, el BOM inicial y los finales de línea de Windows.

Código comentado:

```
export function parsearCSV(texto) {
  if (texto.charCodeAt(0) === 0xfeff) texto = texto.slice(1);   // quita el BOM
  const filas = [];
  let fila = [];
  let celda = '';
  let entreComillas = false;
  for (let i = 0; i < texto.length; i++) {
    const c = texto[i];
    if (entreComillas) {
      if (c === '"') {
        if (texto[i + 1] === '"') { celda += '"'; i++; }   // "" es una comilla
        else entreComillas = false;
      } else celda += c;
    } else if (c === '"') entreComillas = true;
    else if (c === ',') { fila.push(celda); celda = ''; }
    else if (c === '\n' || c === '\r') {
      if (c === '\r' && texto[i + 1] === '\n') i++;        // CRLF
      fila.push(celda); filas.push(fila); fila = []; celda = '';
    } else celda += c;
  }
  if (celda !== '' || fila.length) { fila.push(celda); filas.push(fila); }
  return filas.filter((f) => f.some((v) => v.trim() !== ''));   // sin filas vacías
}
```

aCSVExcel **Descripción:** Prepara el archivo para Excel (punto y coma y BOM) y pone un apóstrofo delante de lo que empiece por `=`, `+`, `-` o `@` para que no se ejecute como fórmula.

Código comentado:

```
export function aCSVExcel(filas) {
  const esc = (v) => {
    let s = v == null ? '' : String(v);
    if (/^[=+\-@\t\r]/.test(s)) s = `'${s}`;          // evita inyección de fórmulas
    return /[";\n\r]/.test(s) ? `"${s.replace(/"/g, '""')}"` : s;
  };
  return '\uFEFF' +   // BOM (en el código va el carácter tal cual)
         filas.map((f) => f.map(esc).join(';')).join('\r\n');
}
```

5.3 datos.js

5.3.1 Ubicación

`js/datos.js`

Descripción general:
Construye el modelo a partir de las pestañas: árboles de categorías y ubicaciones, líneas de existencias, totales y avisos. También filtra, ordena y calcula estadísticas.

5.3.2 Funciones

arbol **Descripción:** Une cada elemento con su padre y detecta padres que no existen y **ciclos** (A dentro de B y B dentro de A), que se avisan y se sacan al primer nivel.

Código comentado:

```
function arbol(lista, nombreHoja, avisos) {
  const porK = new Map(lista.map((n) => [n.k, n]));
  lista.forEach((n) => { n.hijos = []; n.padre = null; });
  lista.forEach((n) => {
    if (!n.padreK) return;
    const p = porK.get(n.padreK);
    if (!p) { avisos.push(`${nombreHoja}, fila ${n._fila}: el padre «${n.padreTexto}» no existe; se muestra en el primer nivel.`); return; }
    n.padre = p;
  });
  lista.forEach((n) => {                          // subir por los padres buscando un ciclo
    const vistos = new Set([n]);
    let p = n.padre;
    while (p) {
      if (vistos.has(p)) {
        avisos.push(`${nombreHoja}, fila ${n._fila}: «${n.nombre}» forma un ciclo de padres; se muestra en el primer nivel.`);
        n.padre = null; break;
      }
      vistos.add(p); p = p.padre;
    }
  });
  lista.forEach((n) => { if (n.padre) n.padre.hijos.push(n); });
  // ... y se guarda la ruta completa de cada uno (n.ruta)
  return porK;
}
```

construirModelo (existencias) **Descripción:** Fragmento que valida cada fila de Existencias: el material debe existir, la cantidad ser un entero y el estado conocido. Las filas repetidas se suman con aviso.

Código comentado:

```
(hojas.existencias || []).forEach((f) => {
  const mat = materiales.get(clave(f.material));
  if (!mat) { avisos.push(`Existencias, fila ${f._fila}: el material «${f.material}» no existe en la hoja Materiales.`); return; }
  const cantidad = entero(f.cantidad);          // NaN si no es un entero >= 0
```

```
  if (Number.isNaN(cantidad)) { /* aviso y se ignora la fila */ return; }
  let estado = estados.get(clave(f.estado));
  // ... si el estado no existe, se avisa y no cuenta como disponible

  const variante = (f.variante || f.talla || '').trim();
  const kLinea = [mat.k, clave(variante), ubicacion ? ubicacion.k : '', estado.k].join
      ('\u0001');
  const previa = combinadas.get(kLinea);
  if (previa) {
    avisos.push(`Existencias, fila ${f._fila}: repite material, variante, ubicación y
        estado de la fila ${previa._fila}; se suman las cantidades.`);
    previa.cantidad += cantidad;
    return;
  }
  const linea = { _fila: f._fila, material: mat, variante, ubicacion, estado, cantidad
      };
  combinadas.set(kLinea, linea);
  mat.lineas.push(linea);
  if (ubicacion) ubicacion.lineas.push(linea);
});
```

filtrar **Descripción:** Filtra por categoría (con subcategorías), estado, disponibilidad y texto. Con varias palabras, las de una o dos letras deben coincidir enteras.

Código comentado:

```
export function filtrar(modelo, { texto = '', categoria = '', estado = '',
    disponibilidad = '' } = {}) {
  const palabras = clave(texto).split(' ').filter(Boolean);
  return [...modelo.materiales.values()].filter((m) => {
    if (!m.visible) return false;
    if (!enCategoria(m, categoria)) return false;
    if (estado && !(m.porEstado.get(modelo.estados.get(estado)) > 0)) return false;
    if (disponibilidad === 'si' && m.disponibles <= 0) return false;
    if (disponibilidad === 'no' && m.disponibles > 0) return false;
    // «estantería a» no debe encontrar todo lo que contiene una «a»
    const exacta = palabras.length > 1;
    return palabras.every((p) => (exacta && p.length <= 2 ? m.tokens.has(p) : m.indice
        .includes(p)));
  });
}
```

urlImagenSegura **Descripción:** Solo acepta fotos de Google Drive (convertidas a miniatura), direcciones `https` o rutas propias sin `...`

Código comentado:

```
export function urlImagenSegura(valor) {
  const s = String(valor ?? '').trim();
  if (!s) return null;
  const drive = s.match(/^https:\/\/drive\.google\.com\/(?:file\/d\/|open\?id=|uc\?(?:
      export=\w+&)?id=)([\w-]{10,})/);
  if (drive) return `https://drive.google.com/thumbnail?id=${drive[1]}&sz=w1200`;
  if (/^https:\/\/[^\s"'<>]+$/i.test(s)) return s;
  if (/^[\p{L}\p{N}_\-./ ]+\.(?:jpe?g|png|webp|gif|avif|svg)$/iu.test(s) && !s.
      startsWith('/') && !s.includes('..')) {
    return s.split('/').map(encodeURIComponent).join('/');
  }
  return null;                                    // cualquier otra cosa se descarta
}
```

5.4 app.js

5.4.1 Ubicación

`js/app.js`

Descripción general:
La interfaz: crea los elementos, lee la ruta de la dirección y pinta cada vista.

5.4.2 Funciones

h y anadir **Descripción:** Crean elementos del DOM. El contenido siempre se añade como **nodo de texto**, así que nada de lo que venga de la hoja se interpreta como HTML (protección XSS).

Código comentado:

```
function h(tag, attrs = {}, ...hijos) {
  const el = document.createElement(tag);
  for (const [k, v] of Object.entries(attrs || {})) {
    if (v == null || v === false) continue;
    if (k === 'class') el.className = v;
    else if (k === 'style') Object.entries(v).forEach(([p, val]) => { if (val != null)
        el.style.setProperty(p, val); });
    else if (k.startsWith('on')) el.addEventListener(k.slice(2), v);
    else if (k === 'dataset') Object.assign(el.dataset, v);
    else el.setAttribute(k, v === true ? '' : v);
  }
  anadir(el, hijos);
  return el;
}

function anadir(el, hijos) {
  hijos.flat(Infinity).forEach((c) => {
    if (c == null || c === false) return;
    el.append(c instanceof Node ? c : document.createTextNode(String(c)));   // texto,
        nunca HTML
  });
}
```

cargar **Descripción:** Enseña el indicador de carga, descarga las hojas, construye el modelo y pinta; si falla, muestra el error con un botón para reintentar.

Código comentado:

```
async function cargar() {
  const main = $('#contenido');
  vaciar(main, h('div', { class: 'cargando', role: 'status' },
    h('div', { class: 'girador', 'aria-hidden': 'true' }), 'Cargando inventario…'));
  try {
    const { hojas, fecha, desdeCopia } = await cargarHojas(CONFIG);
    estado.modelo = construirModelo(hojas);
    estado.fecha = fecha;
    estado.desdeCopia = desdeCopia;
    pintar();
  } catch (e) {
    console.error(e);
    vaciar(main, aviso('error', h('strong', {}, 'No se ha podido cargar el inventario.
        '), e.message || '',
      h('div', {}, h('button', { class: 'boton', type: 'button', onclick: cargar }, '
          Reintentar'))));
  }
}
```

5.5 qr.js

5.5.1 Ubicación

`js/qr.js`

Descripción general:
Generador de códigos QR propio (modo byte, corrección de errores nivel M), con su cálculo Reed-Solomon, el dibujo de los patrones y la elección de la máscara.

5.5.2 Funciones

generarQR (inicio) **Descripción:** Elige la versión más pequeña en la que cabe el texto y construye el flujo de bits: modo byte, longitud, datos, terminador y relleno.

Código comentado:

```
export function generarQR(texto) {
  const bytes = Array.from(new TextEncoder().encode(texto));
  let ver = 0;
  for (let v = 1; v <= 40; v++) {                    // la versión más pequeña que sirve
    const bitsCuenta = v < 10 ? 8 : 16;
    if (4 + bitsCuenta + bytes.length * 8 <= codewordsDatos(v) * 8) { ver = v; break; }
  }
  if (!ver) throw new Error('Texto demasiado largo para un código QR');

  const bits = [];
  const anadir = (valor, n) => { for (let i = n - 1; i >= 0; i--) bits.push((valor >>> i) & 1); };
  anadir(0b0100, 4);                                 // modo byte
  anadir(bytes.length, ver < 10 ? 8 : 16);
  bytes.forEach((b) => anadir(b, 8));
  const capacidad = codewordsDatos(ver) * 8;
  anadir(0, Math.min(4, capacidad - bits.length));   // terminador
  anadir(0, (8 - (bits.length % 8)) % 8);
  for (let relleno = 0xec; bits.length < capacidad; relleno ^= 0xec ^ 0x11) anadir(relleno, 8);
  // ... después: bloques con Reed-Solomon, intercalado, patrones y máscara
}
```

qrSVG **Descripción:** Dibuja la matriz como un SVG con un solo `path`, para que se vea nítido al imprimirlo a cualquier tamaño.

Código comentado:

```
export function qrSVG(texto, { margen = 4, titulo = '' } = {}) {
  const m = generarQR(texto);
  const tam = m.length + margen * 2;
  const svg = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
  svg.setAttribute('viewBox', `0 0 ${tam} ${tam}`);
  svg.setAttribute('shape-rendering', 'crispEdges');
  // ... fondo blanco y título accesible
  let d = '';
  m.forEach((fila, y) => fila.forEach((osc, x) => {
    if (osc) d += `M${x + margen} ${y + margen}h1v1h-1z`;   // un cuadrado por módulo negro
  }));
  // ... se añade el path con d
  return svg;
}
```